

THREAT INTELLIGENCE SYSTEM

Project Team Members

Sr. No	Name	Registration No
1	Muhammad Madni	B23F0309SE065
2	Muhammad Wasif	B23F0396SE006
3	Muhammad Zohaib	B23F0515SE005
4	Duad Latif	B23F0195SE011
5	Haneeza Liaqat Awan	B23F0493SE010

1. Introduction

The **Threat Intelligence System** is a full-stack web-based application developed to collect, manage, and analyze cybersecurity threats using Indicators of Compromise (IOCs). These indicators include IP addresses, domain names, severity levels, and sources of threats.

The system is designed to help security analysts and organizations track potential threats efficiently and maintain a structured database for further analysis. It integrates modern technologies such as **Spring Boot, MySQL, and Apache Kafka** to ensure scalability, performance, and real-time processing.

2. Objectives

The main objectives of this project are:

- To design and develop a system for managing threat intelligence data
 - To store Indicators of Compromise (IOC) in a structured database
 - To provide RESTful APIs for communication between frontend and backend
 - To implement real-time data streaming using Kafka
 - To build a simple and interactive web dashboard
 - To demonstrate full-stack development using modern technologies
-

3. Scope of the System

The system is designed for educational and foundational cybersecurity use. It allows:

- Manual entry of threat data
- Storage and retrieval of IOC records
- Real-time messaging using Kafka
- Visualization of stored data through a web interface

Future extensions can make it suitable for enterprise-level threat monitoring systems.

4. Technologies Used

4.1 Backend

- Java 17
- Spring Boot
- Spring Web
- Spring Data JPA

4.2 Database

- MySQL

4.3 Messaging System

- Apache Kafka
- Zookeeper

4.4 Frontend

- HTML
- CSS
- JavaScript

4.5 Tools & Platforms

- Maven
 - Docker (for Kafka & Zookeeper)
 - IntelliJ IDEA / VS Code
-

5. System Architecture

The system follows a **layered architecture**, ensuring separation of concerns and scalability.

Architecture Layers:

1. **Presentation Layer (Frontend)**
 - User interacts via dashboard
2. **Controller Layer (REST API)**
 - Handles HTTP requests
3. **Service Layer**
 - Contains business logic
4. **Repository Layer**
 - Communicates with database
5. **Database Layer (MySQL)**

- Stores IOC data
-

Kafka Integration Flow:

- Kafka Producer → Sends messages
 - Kafka Topic → Stores messages
 - Kafka Consumer → Processes messages
-

6. System Modules

6.1 IOC Entity Module

This module defines the structure of threat data.

Attributes:

- IP Address
 - Domain Name
 - Severity Level
 - Source
-

6.2 REST API Module

Provides endpoints for interaction.

Method	Endpoint	Description
POST	/ioc	Save IOC data
GET	/ioc	Retrieve all IOC data

6.3 Service Layer

- Implements business logic
- Validates and processes data

- Coordinates between controller and repository
-

6.4 Repository Layer

- Uses Spring Data JPA
 - Performs CRUD operations
 - Connects directly to MySQL database
-

6.5 Kafka Module

- Producer sends threat data to Kafka topic
 - Consumer listens and processes messages
 - Enables real-time threat handling
-

6.6 Frontend Dashboard

A simple UI that allows:

- Input of IOC data
 - Viewing stored data in table format
 - Interaction with backend APIs
-

7. Database Design

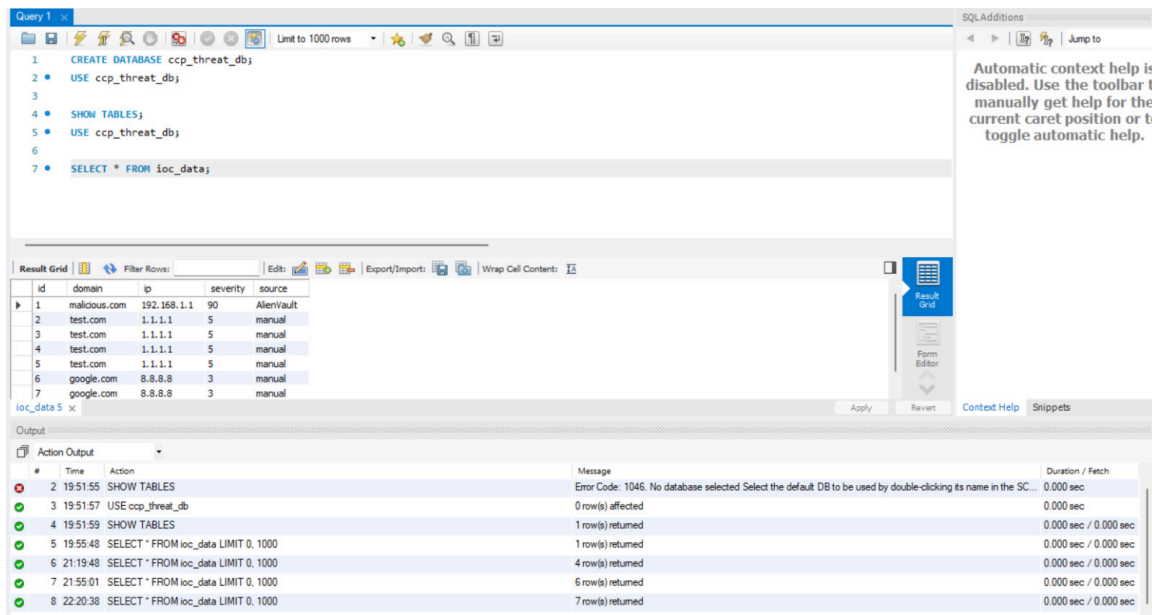
Table: **ioc_data**

Field	Type	Description
id	Long	Primary Key
ip	String	IP Address

domain String Domain Name

severity int Threat Severity

source String Data Source



8. System Workflow

1. User enters IOC data through dashboard
2. Data is sent to Spring Boot REST API
3. Backend processes and stores data in MySQL
4. Kafka producer sends message to topic
5. Kafka consumer processes message
6. Stored data is displayed on frontend

9. Features of the System

- Add and manage threat intelligence data
- View all stored threats
- REST API-based communication
- Real-time data streaming with Kafka
- Persistent data storage using MySQL
- Simple and user-friendly interface

10. Sample Output

The screenshot shows a web browser window with the address bar set to localhost:8080. The browser's bookmark bar includes links to Gmail and Maps. The main content area displays the 'Threat Intelligence Dashboard'.

Threat Intelligence Dashboard

Add IOC

IP: Domain: Severity: Source:

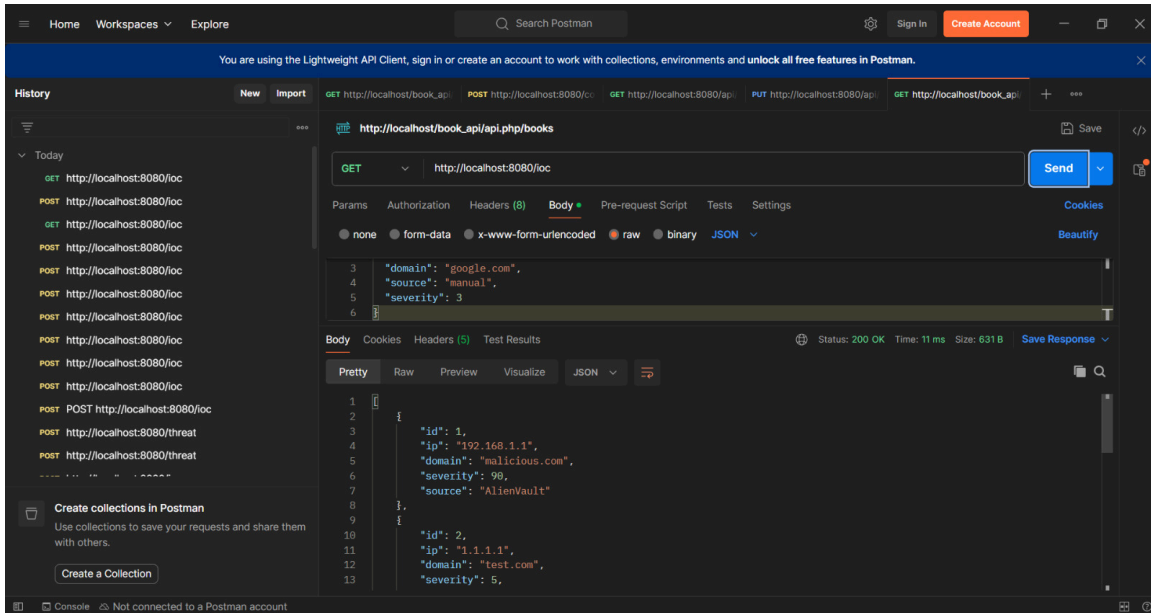
All IOC Data

ID	IP	Domain	Severity	Source
1	192.168.1.1	malicious.com	90	AlienVault
2	1.1.1.1	test.com	5	manual
3	1.1.1.1	test.com	5	manual
4	1.1.1.1	test.com	5	manual
5	1.1.1.1	test.com	5	manual
6	8.8.8.8	google.com	3	manual

```
@503c93719012:~
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\loppom> docker exec -it kafka bash
[appuser@503c93719012 ~]$ kafka-console-consumer --topic threat-topic --from-beginning --bootstrap-server localhost:9092
com.ecp.threat_system.entity.IOCData@581a89b
{"id":5,"ip":"1.1.1.1","domain":"test.com","severity":5,"source":"manual"}
{"id":6,"ip":"8.8.8.8","domain":"google.com","severity":3,"source":"manual"}
{"id":7,"ip":"8.8.8.8","domain":"google.com","severity":3,"source":"manual"}
```



11. Testing

The system can be tested using:

- **Postman** (for API testing)
- **Browser** (for frontend testing)
- **MySQL Workbench** (for database verification)

12. Limitations

- No authentication system
- Manual data entry only
- No real-time UI updates

- Basic frontend design
-

13. Future Improvements

- Real-time updates using WebSocket
 - AI/ML-based threat classification
 - JWT-based authentication and security
 - Advanced dashboard with analytics
 - Role-based access control
 - Integration with external threat APIs
-

14. Conclusion

The Threat Intelligence System successfully demonstrates the integration of backend, database, messaging system, and frontend into a complete full-stack application.

It provides a strong foundation for building advanced cybersecurity tools and highlights the importance of real-time data processing and structured threat management systems.

This project enhances understanding of:

- Spring Boot development
 - Database integration
 - Kafka messaging systems
 - Full-stack architecture
-

15. References

- Spring Boot Documentation
- Apache Kafka Documentation
- MySQL Documentation
- Java Official Documentatio